文本分类实验报告

课程：自然语言处理

作业：作业一 —— 文本分类实现

姓名：梁杰

学号：2313771

日期：2026年10月12日

一、实验目的

本次实验旨在理解和掌握不同文本表示方法在文本分类任务中的基本原理与实际效果，具体目标包括：

1. 理解 Bag-of-Words、Word Embedding、预训练语言模型三类文本表示方法的基本思想；
2. 掌握文本分类任务的基本实验流程；
3. 熟悉 Logistic Regression 在文本分类中的使用；
4. 掌握 Word2Vec、GloVe 等词向量的基本使用方法；
5. 熟悉 BERT 类预训练语言模型的 Fine-tuning 流程；
6. 能够使用 Accuracy 和 Macro-F1 对文本分类模型进行评价；
7. 能够对不同文本表示方法的性能进行基本比较和分析。

二、数据集与预处理

2.1 数据集

- **NYT 新闻数据集**：本次实验的主数据集，包含 `text`（新闻文本）和 `label`（新闻类别）两列，共 11519 条样本。
- **AG News 数据集**：用于训练 Word2Vec 词向量，共 90000 条样本，仅使用 `text` 列。

2.2 数据划分

对 NYT 数据集先随机打乱（`random_state=42`），再按 80% / 10% / 10% 划分为训练集、验证集和测试集：

集合	样本数	占比
Training Set	9215	80%
Validation Set	1152	10%
Test Set	1152	10%

所有实验均使用同一划分，保证不同方法之间具有可比性。最终模型性能统一在 NYT Test Set 上评价。

2.3 分词

所有方法统一使用 NLTK 的 `word_tokenize` 进行分词，配合小写化和非字母数字字符清洗：

代码块

```
def tokenize(text):
    text = str(text).lower()
    text = re.sub(r"[^a-z0-9\s]", " ", text)
    return word_tokenize(text)
```

2.4 评价指标

- **Accuracy**：分类正确的样本数占全部测试样本的比例。
- **Macro-F1**：分别计算每个类别的 F1 Score，再对所有类别取平均，对类别不均衡场景更鲁棒。

三、Task 1：Bag of Words（30 分）

本部分使用两种 Bag-of-Words 文本表示方法，统一使用 Logistic Regression（`max_iter=1000`）作为分类模型。

3.1 Binary Bag of Words

原理：构建全训练语料的词表 V，每篇文档表示为一个 $|V|$ 维向量：

$$\mathbf{x}_i = \begin{cases} 1, & \text{若词表中第 } i \text{ 个词出现在文档中} \\ 0, & \text{否则} \end{cases}$$

只考虑单词是否出现，不考虑出现次数。实现上使用 `CountVectorizer(binary=True)`。

3.2 Word Frequency

原理：仍使用词表构建文档向量，但记录每个词在当前文档中的出现次数：

$$\mathbf{x} = (c_1, c_2, \ldots, c_{|V|})$$

其中 c_i 表示词表中第 i 个词在当前文档中的出现次数。实现上使用 `CountVectorizer(binary=False)`。

3.3 实验结果

方法	Validation Accuracy	Validation Macro-F1	Test Accuracy	Test Macro-F1	训练耗时
Binary Bag of Words	0.9835	0.9596	0.9887	0.9728	18.4s
Word Frequency	0.9809	0.9508	**0.9922**	**0.9820**	22.1s

3.4 结果分析

- 两种 BOW 表示在 NYT 新闻分类上都取得了非常好的效果，Test Accuracy 均超过 98%，说明新闻类别具有明显的词汇特征。
- Word Frequency 在 Test 集上略优于 Binary BOW（Accuracy 0.9922 vs 0.9887，Macro-F1 0.9820 vs 0.9728），因为词频包含更多信息量，Logistic Regression 可以从高频词中获取更有区分度的权重。
- 但验证集上 Binary BOW 的 Accuracy 略高，说明两种表示在泛化能力上的差异不大，Test 集上的差异可能带有一定随机性。
- BOW 方法的主要局限是忽略词序和上下文语义，但在类别特征明确的新闻分类任务中，这种局限影响有限。

四、Task 2：Word2Vec（50 分）

本部分使用基于 Word Embedding 的方式表示文档，统一使用 **100 维词向量**，分类器仍使用 Logistic Regression。文档向量通过对文档内所有有效单词的词向量取平均得到：

$$\mathbf{e}(\mathbf{d}) = \frac{1}{n}\sum_{i=1}^{n}\mathbf{e}(\mathbf{w}_i)$$

4.1 Pre-trained GloVe

使用公开的 `glove.6B.100d.txt` 预训练词向量。加载后构建词向量字典，对每篇 NYT 文档：

1. 分词；
2. 找到每个词对应的 GloVe 向量；
3. 对所有有效词向量取平均，得到 100 维文档向量；
4. 用 Logistic Regression 完成分类。

4.2 Word2Vec Trained on AG News

使用 `gensim.models.Word2Vec` 在 AG News 的 90000 条文本上训练 100 维 Word2Vec 模型，主要参数：

代码块

```
vector_size=100, window=5, min_count=1, workers=4, epochs=5, seed=42
```

训练完成后，将模型转为词向量字典，对 NYT 文档以同样方式取平均并分类。

4.3 Word2Vec Trained on NYT

在 **NYT 训练集**（9215 条）上训练 100 维 Word2Vec 模型，避免使用验证集和测试集造成数据泄露。其余流程同上。

4.4 实验结果

方法	Validation Accuracy	Validation Macro-F1	Test Accuracy	Test Macro-F1	训练耗时
Pre-trained GloVe (100d)	0.9766	0.9447	0.9783	0.9491	27.5s
Word2Vec on AG News	0.9748	0.9389	0.9774	0.9495	36.5s
Word2Vec on NYT	0.9748	0.9365	0.9792	0.9502	41.2s

4.5 结果分析

- 三种 Word Embedding 方法的性能非常接近，Test Accuracy 均在 97.7%–97.9% 之间，Macro-F1 在 0.949–0.950 之间，差异在合理随机波动范围内。
- **GloVe** 由于在大规模通用语料上预训练，词向量质量较高，验证集上表现最好（Macro-F1 0.9447）。
- **Word2Vec on AG News** 使用跨领域数据训练，虽然 AG News 与 NYT 同属新闻领域，但类别体系和用词习惯不完全一致，性能略低于 GloVe。
- **Word2Vec on NYT** 由于训练语料与目标任务同分布，Test 集表现最好（Accuracy 0.9792，Macro-F1 0.9502），但训练集较小（仅 9215 条），词向量覆盖度有限。
- 整体上，Word Embedding 方法略低于 BOW 方法，可能原因是**对文档内所有词向量取平均会丢失词序和部分语义信息**，而 BOW 在高维稀疏空间中保留了每个词的出现特征，在类别特征明确的新闻分类任务中反而更有效。

4.6 实验中遇到的问题与解决过程

在实现 Task 2 的过程中遇到了一处接口不一致的问题，这里如实记录：

问题现象：运行 `main.py` 到 Task 2 的 GloVe 阶段时，程序报错：

```
代码块
1   ValueError: not enough values to unpack (expected 4, got 3)
```

原因分析：`main.py` 中使用 `val, test, _, _ = run_glove(...)` 解包 4 个返回值，而 `task2_word2vec.py` 中 `_train_and_eval_vectors` 以及三个 `run_*` 函数实际只返回了 3 个值。这是因为 Task 1 的 BOW 方法返回 `(val, test, clf, vectorizer)` 共 4 个值，而 Task 2 编写时未保持一致的返回结构。

解决方案：修改 `task2_word2vec.py`，让 `_train_and_eval_vectors` 统一返回 `(val_metrics, test_metrics, clf, None)`，三个 `run_*` 函数相应返回 4 个值。修改后 Task 1 和 Task 2 的接口保持一致，主程序无需修改即可连续运行。

这次问题提示我们在多任务共用一个主流程时，**各模块的返回接口应保持统一约定**，或在 `main.py` 中根据实际返回值数量分别处理，避免因解包不匹配导致中断。

五、Task 3：Pre-trained Neural Models（20 分）

5.1 任务要求

按作业要求，本部分需微调 `google-bert/bert-base-uncased`，超参数如下：

- Pre-trained model: BERT-base-uncased
- Maximum sequence length: 64
- Number of epochs: 3
- 在 NYT Test Set 上报告 Accuracy 和 Macro-F1

5.2 实现方案

`src/task3_bert.py` 中已经完成了完整的 BERT 微调实现，主要结构如下：

（1）数据集封装

代码块

```
class NYTDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=64):
        ...
    def __getitem__(self, idx):
        enc = self.tokenizer(
            str(self.texts[idx]),
            truncation=True,
            padding="max_length",
            max_length=self.max_len,
            return_tensors="pt"
        )
        item = {k: v.squeeze(0) for k, v in enc.items()}
        item["labels"] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item
```

将每条文本编码为固定长度 64 的 `input_ids`、`attention_mask`，标签转为 `torch.long`。

（2）标签映射

代码块

```
def build_label_map(train_df, val_df, test_df):
    labels = sorted(train_df["label"].unique().tolist())
    label2id = {label: i for i, label in enumerate(labels)}
    id2label = {i: label for label, i in label2id.items()}
    ...
```

将原始标签字符串映射为 0 到 num_labels-1 的整数，供分类头使用。

（3）评价函数

代码块

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {
        "accuracy": accuracy_score(labels, preds),
        "macro_f1": f1_score(labels, preds, average="macro")
    }
```

同时报告 Accuracy 和 Macro-F1。

（4）训练配置

代码块

```
training_args = TrainingArguments(
    output_dir="outputs/bert",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="macro_f1",
    learning_rate=2e-5,
    weight_decay=0.01,
    report_to="none"
)
```

使用 HuggingFace `Trainer` 完成微调，训练 3 个 epoch，每轮在验证集上评估，根据 Macro-F1 选择最优模型。

（5）模型加载

```
代码块
1   tokenizer = BertTokenizer.from_pretrained(model_path, local_files_only=True)
2   model = BertForSequenceClassification.from_pretrained(
3       model_path,
4       num_labels=len(label2id),
5       id2label=id2label,
6       label2id=label2id,
7       local_files_only=True
8   )
```

其中 `model_path` 默认为本地目录 `./bert-base-uncased`，`local_files_only=True` 表示从本地加载、不发起网络请求。

5.3 测试失败及原因说明

问题现象：在尝试从 HuggingFace 官方源和镜像 `hf-mirror.com` 下载 BERT-base-uncased 模型时，反复出现 DNS 解析失败：

```
代码块
1   [Errno 11002] getaddrinfo failed while requesting HEAD
    https://huggingface.co/...
```

更换镜像后，小文件（`config.json`、`vocab.txt`、`tokenizer.json`）可以成功下载，但大文件 `model.safetensors`（约 440MB）在下载过程中被重定向到 `cas-bridge.xethub.hf.co`，随后再次出现：

```
代码块
```

```
curl: (6) Could not resolve host: cas-bridge.xethub.hf.co
```

原因分析：

1. HuggingFace 自 2025 年起将大模型文件迁移到 **XetHub CDN**（`cas-bridge.xethub.hf.co`）分发，镜像站对元数据请求生效，但大文件仍会 302 重定向到 Xet CDN。
2. 当前网络环境无法解析 `cas-bridge.xethub.hf.co` 域名，导致大文件无法下载。
3. 尝试过的方式包括：`HF_ENDPOINT=https://hf-mirror.com` 镜像、`HF_HUB_DISABLE_XET=1` 禁用 Xet、`huggingface-cli` / `hf` CLI 下载、`curl.exe` 直接下载、浏览器手动下载，均由于同一 DNS 解析失败问题未能完成。

处理结果：

- Task 1 和 Task 2 均已正常完成，Test 集上的 Accuracy 和 Macro-F1 均已获得；
- Task 3 的**代码实现完整**（见 5.2 节及 `src/task3_bert.py`），符合作业要求的模型、序列长度和训练轮数设置，只是由于网络环境限制无法下载模型权重而未能实际运行；
- 若将 `bert-base-uncased` 的模型文件手动放入 `./bert-base-uncased/` 目录，`task3_bert.py` 中的 `local_files_only=True` 即可直接从本地加载并完成训练。

5.4 Task 3 结果

由于上述网络限制，Task 3 未能获得实际的测试结果，故本节仅列出实现方案，不提供 Accuracy 和 Macro-F1 数值。

六、总体结果对比

方法	Validation Accuracy	Validation Macro-F1	Test Accuracy	Test Macro-F1
Binary Bag of Words	0.9835	0.9596	0.9887	0.9728
Word Frequency	0.9809	0.9508	0.9922	0.9820
Pre-trained GloVe (100d)	0.9766	0.9447	0.9783	0.9491
Word2Vec on AG News	0.9748	0.9389	0.9774	0.9495
Word2Vec on NYT	0.9748	0.9365	0.9792	0.9502
BERT-base-uncased	网络限制，未能运行	—	—	—

七、结论与总结

1. **BOW 方法在 NYT 新闻分类上表现优异**：Binary BOW 和 Word Frequency 的 Test Accuracy 均超过 98%，说明新闻类别与词汇特征高度相关，稀疏高维表示在该任务上非常有效。
2. **Word Embedding 方法性能略低于 BOW**：三种 Word Embedding 方法（GloVe、Word2Vec on AG、Word2Vec on NYT）的 Test Accuracy 均在 97.7%–97.9% 之间，Macro-F1 在 0.949–0.950 之间，差异不大。对文档内所有词向量取平均虽然能压缩维度、引入语义信息，但也丢失了词序和部分区分性特征。
3. **预训练词向量与自训练词向量差异有限**：GloVe 因为大规模通用语料预训练，在验证集上表现略好；Word2Vec on NYT 由于与目标任务同分布，在 Test 集上表现最好，但整体差异在合理随机波动范围内。
4. **BERT 代码实现完整，但受网络限制未能运行**：`task3_bert.py` 完整实现了 BERT 微调流程，采用 `bert-base-uncased`、最大序列长度 64、3 个 epoch、AdamW 优化器和 Macro-F1 早停选择，符合作业要求。由于 HuggingFace Xet CDN 的 DNS 解析问题，模型权重无法下载，故 Task 3 未获得实际结果。
5. **接口一致性的重要性**：Task 2 中由于返回值与 `main.py` 解包不匹配引发 `ValueError`，提醒我们在多模块协作时需严格统一接口约定。

八、代码运行说明

8.1 环境

- Python 3.10
- 依赖：`pandas`、`numpy`、`scikit-learn`、`nltk`、`gensim`、`torch`、`transformers`、`datasets`、`tqdm`

8.2 数据放置

代码块

```
text-classification/
├── data/
│   ├── nyt.csv
│   └── ag.csv
├── glove/
│   └── glove.6B.100d.txt
├── src/
│   ├── data_utils.py
│   ├── task1_bow.py
│   ├── task2_word2vec.py
│   ├── task3_bert.py
│   └── main.py
└── requirements.txt
```

8.3 运行方式

代码块

```
cd "E:\校用\4\自然语言\text-classification"
```

```
.venv\Scripts\activate
python -m src.main
```

当前 `main.py` 已注释掉 Task 3 调用，运行后会依次完成 Task 1 和 Task 2 并打印结果。若日后模型文件下载成功，取消 `main.py` 中 Task 3 相关注释即可。

附录：GitHub 仓库

https://github.com/aphelios-lj/NLP
